Convolution và pooling

Một bộ dò nét dùng lại trên cả ảnh

Phase 5 · Bài 5.1

Nét phấn dịch chỗ

MLP trải cả ảnh thành một vector.

Khi nét đổi vị trí, MLP có thể phải học lại nhiều trọng số.

Ta muốn đọc cửa sổ nhỏ.

Cùng kernel, nhiều vị trí.

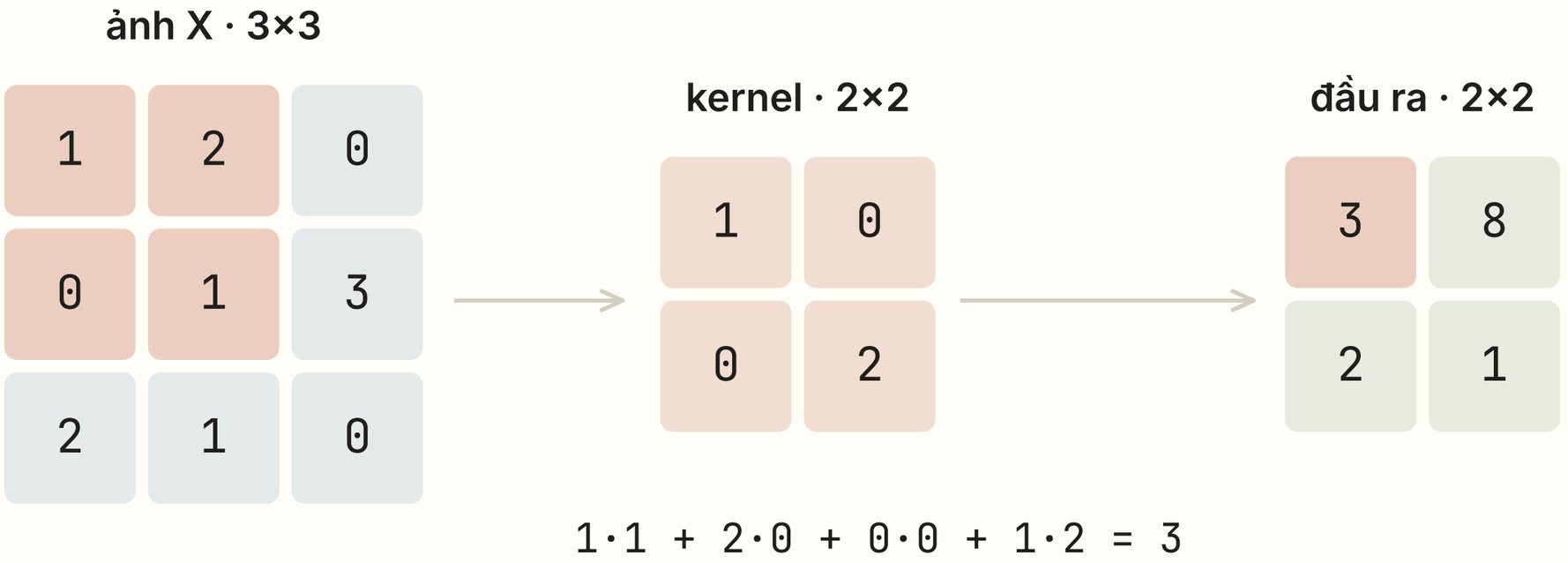
Mục tiêu buổi này

- Tính convolution bằng tay.
- Kiểm shape, kích thước, số tham số.
- Đọc pooling và receptive field.
- Biết khi nào phải giữ vị trí.

01

Đọc một cửa sổ

Bốn phép nhân, một kết quả



Cửa sổ 2×2 đầu tiên (tô cam) nhân từng ô với kernel rồi cộng: 1·1 + 2·0 + 0·0 + 1·2 = 3, ô đầu của đầu ra; trượt kernel qua bốn vị trí được 3, 8, 2, 1.

Không lật kernel

Convolution toán học lật kernel $180°$.

`Conv2d` làm cross-correlation: nhân thẳng.

Kernel học được: lật hay không như nhau.

Kernel đề cho sẵn: **tính không lật**.

HỎI NHANH

Hàng ảnh `[0, 0, 5, 5]` có một cạnh chuyển từ tối sang sáng. Kernel `[-1, 1]`, stride 1, không padding. `Conv2d` của PyTorch cho ra kết quả nào?

A `[0, 5, 0]`

B `[0, -5, 0]`

C `[0, 0, 5]`

D `[0, 5, 0, 0]`

HỎI NHANH

Ảnh xám 64×64, kernel 3×3, không padding, stride 1, đầu ra 62×62. Nếu mỗi vị trí đầu ra có một kernel riêng thay vì dùng chung, lớp này cần bao nhiêu trọng số, bỏ qua bias?

A 9

B 3844

C 34 596

D 36 864

PHỎNG THEO CMU 10-601 · Exam 3 Practice · Q1.4(a)

Tính tay ô đầu

Ô trên trái là $1 \times 1 + 2 \times 0 + 0 \times 0 + 1 \times 2 = 3$.

Ô bên phải là $2 \times 1 + 0 \times 0 + 1 \times 0 + 3 \times 2 = 8$.

Code đọc từng cửa sổ

`image` và `kernel` đã được tạo ở ví dụ tính tay; hàm dưới chỉ đọc các cửa sổ nằm trọn trong ảnh (không padding).

Không lật kernel: đây là cross-correlation, cũng là quy ước của `Conv2d` trong bài.

```
def correlate_valid(image, kernel):
    height, width = image.shape
    kh, kw = kernel.shape
    result = np.zeros((height - kh + 1, width - kw + 1))
    for row in range(result.shape[0]):
        for col in range(result.shape[1]):
            window = image[row:row + kh, col:col + kw]
            result[row, col] = np.sum(window * kernel)
    return result
```

Kết quả để đối chiếu

```
feature_map = correlate_valid(image, kernel)
print(feature_map)
```

KẾT QUẢ

```
[[3. 8.]
 [2. 1.]]
```

Ô 8 lớn nhất: cửa sổ ấy có số lớn trên đường chéo, đúng chỗ kernel có trọng số dương.

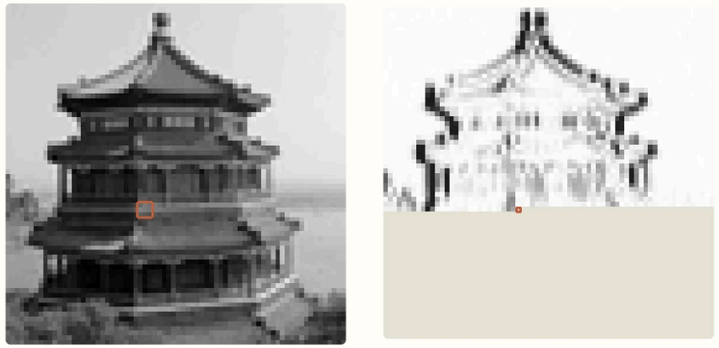
Một kernel quét cả ảnh



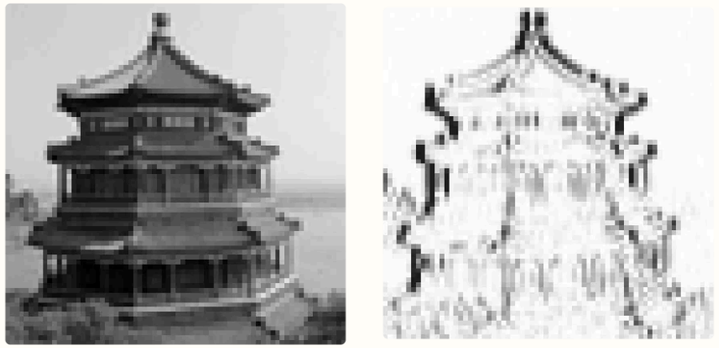
cửa sổ ở hàng 3, cột 6



cửa sổ ở hàng 20, cột 29



cửa sổ ở hàng 43, cột 29



xong: 70 × 70 ô, mỗi ô dùng cùng 9 trọng số

Kernel nét dọc trượt qua ảnh tháp; mỗi vị trí cho một ô của bảng đầu ra.

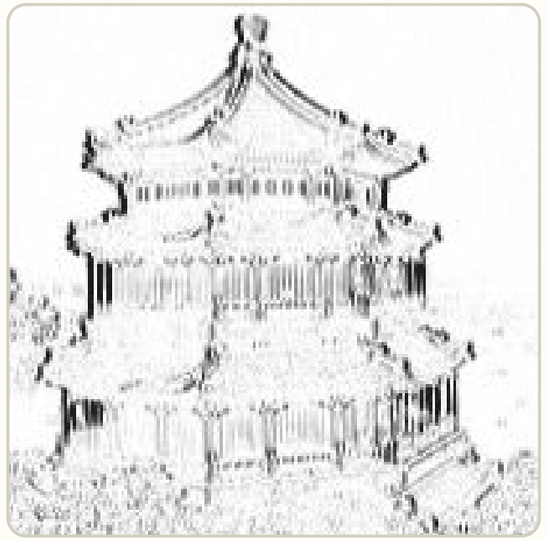
Kernel dò nét trên ảnh thật

ảnh xám



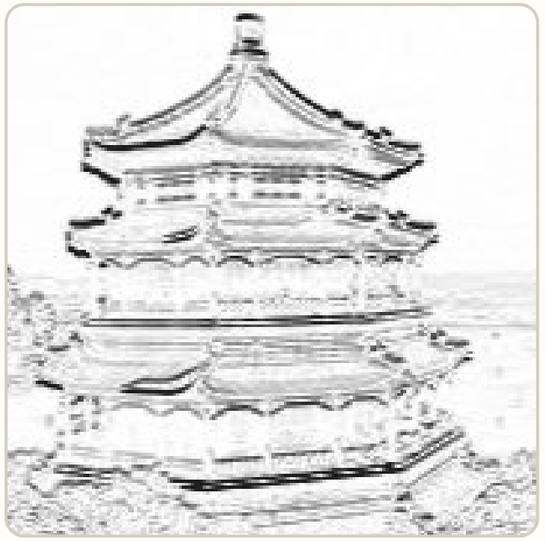
cùng 9 trọng số trượt qua mọi vị trí

kernel nét dọc



-1	0	1
-2	0	2
-1	0	1

kernel nét ngang



-1	-2	-1
0	0	0
1	2	1

Ảnh thật qua hai kernel 3×3 đặt tay: kernel nét dọc làm sáng các cạnh dựng đứng của tháp, kernel ngang làm sáng mái và lan can.

HỎI NHANH

Ta muốn một kernel 2×2 cho số dương với cửa sổ nét chéo $\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$, nhưng cho 0 với mọi vùng nền đều màu $\begin{bmatrix} c & c \\ c & c \end{bmatrix}$. Chọn kernel nào?

A $\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$

B $\begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}$

C $\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$

D $\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$

02

Đọc thưa, đọc sát biên

Ba cách đổi cửa sổ

Padding thêm vành quanh ảnh.

Stride quyết định mỗi lần dịch bao nhiêu ô.

Dilation giãn vị trí kernel.

Kernel 3, dilation 2 phủ 5 ô.

Kích thước một chiều

$$H_{\text{out}} = \left\lfloor \frac{H + 2p - d(k-1) - 1}{s} \right\rfloor + 1$$

p là padding, s là stride, còn d là dilation.

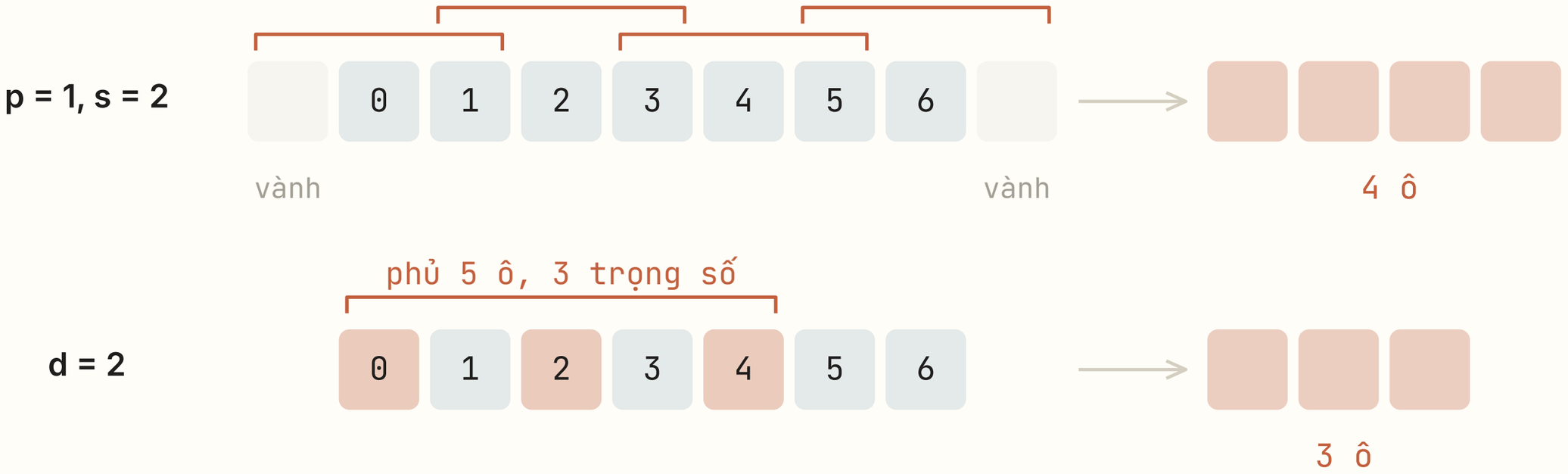
Giữ kích thước khi $s = 1$: $2p = d(k-1)$.

Ví dụ kích thước

$$H = 7,\ k = 3,\ p = 1,\ s = 2,\ d = 1$$

$$H_{\text{out}} = \left\lfloor \frac{7 + 2 - (3 - 1) - 1}{2} \right\rfloor + 1 = \lfloor 6/2 \rfloor + 1 = 4.$$

Padding, stride, dilation trên một chiều



Kernel 3 trên chiều dài 7: thêm vành 0 hai đầu và dịch 2 ô thì cửa sổ bắt đầu ở 0, 2, 4, 6, ra 4 ô; dilation 2 đọc cách một ô, phủ 5 ô nên chỉ còn 3 vị trí.

HỎI NHANH

Ảnh dài 8 ô, kernel 3, padding 0, stride 2, dilation 1. Đầu ra dài bao nhiêu?

A 2

B 3

C 4

D 6

HỎI NHANH

Conv có kernel 3, dilation 2, stride 1. Padding đối xứng bằng bao nhiêu thì giữ nguyên chiều dài đầu vào?

A 1

B 2

C 3

D 4

Nhiều kênh

Input có shape (N, C_{in}, H, W).

Một kernel tạo một kênh đầu ra và đọc toàn bộ C_{in} kênh đầu vào.

RGB, 8 kênh đầu ra, kernel 3×3:

$8(3 \cdot 9 + 1) = 224$ tham số.

Số tham số không phụ thuộc H, W của ảnh.

Đếm bằng thư viện

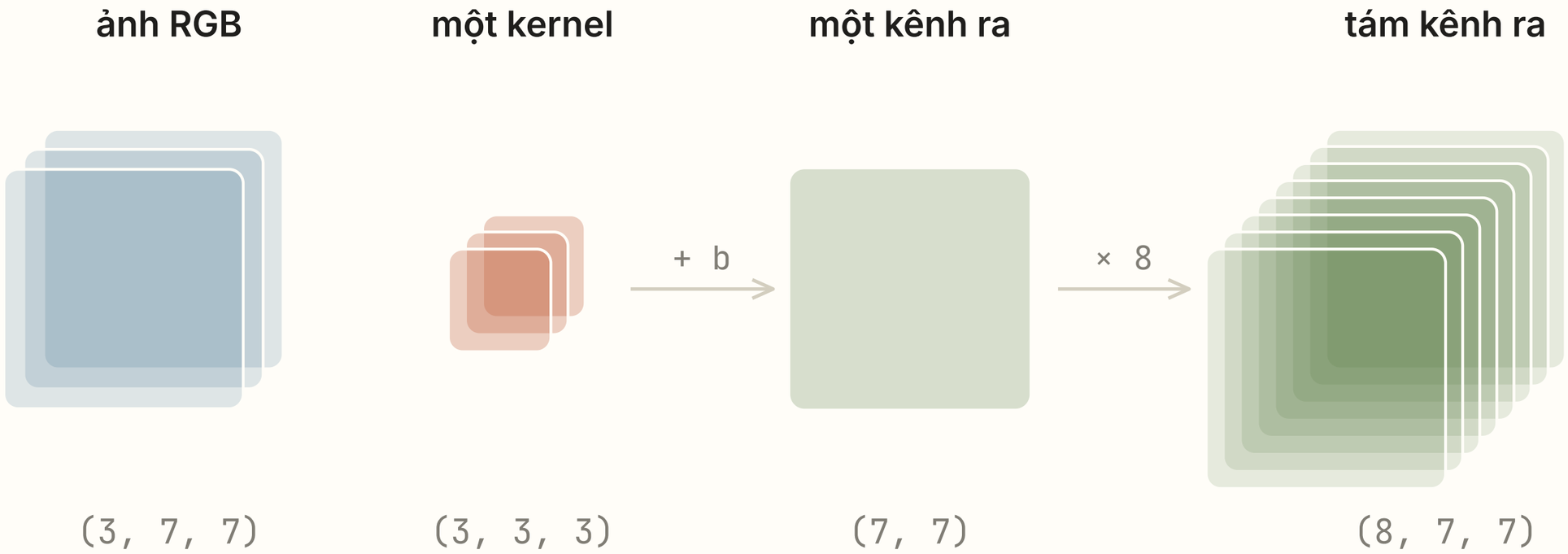
```
rgb_layer = nn.Conv2d(3, 8, 3, padding=1)
print(sum(parameter.numel() for parameter in rgb_layer.parameters()))
print(tuple(rgb_layer(torch.zeros(2, 3, 7, 7)).shape))
```

KẾT QUẢ

```
224
(2, 8, 7, 7)
```

Padding 1 giữ 7×7; hai ảnh, tám kênh.

Một kernel đọc mọi kênh vào



$$8 \cdot (3 \cdot 9 + 1) = 224 \text{ tham số}$$

Một kernel đọc cả ba kênh RGB, cộng lại thêm bias thành một kênh ra; tám kernel độc lập cho tám kênh, tổng 8·(3·9 + 1) = 224 tham số dù ảnh lớn hay nhỏ.

HỎI NHANH

`nn.Conv2d(16, 32, kernel_size=(3, 5))` có `weight.shape` là gì?

A `(16, 32, 3, 5)`

B `(32, 3, 5)`

C `(32, 16, 3, 5)`

D `(32, 16, 5, 3)`

HỎI NHANH

Lớp `nn.Conv2d(3, 8, 3, padding=1)` chạy trên ảnh 7×7, rồi chạy trên ảnh 70×70. Số tham số và số phép nhân thay đổi thế nào?

A Tham số vẫn là 224, phép nhân tăng 100 lần

B Tham số vẫn là 224, phép nhân cũng giữ nguyên

C Tham số tăng 100 lần, phép nhân tăng 100 lần

D Tham số tăng 10 lần, phép nhân tăng 10 lần

03

Tổng hợp và vùng nhìn

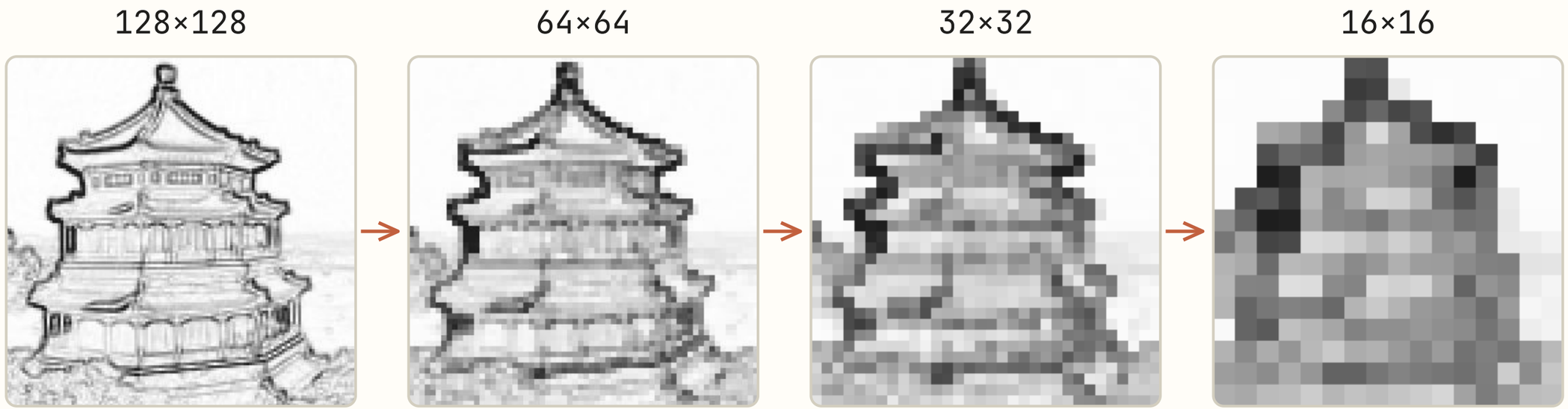
Pooling trên bảng kết quả

$$\begin{bmatrix} 3 & 8 \\ 2 & 1 \end{bmatrix}$$

Max pooling trả về **8**. Average pooling trả về **3.5**.

Không có trọng số học. Global pooling: cả map còn một số.

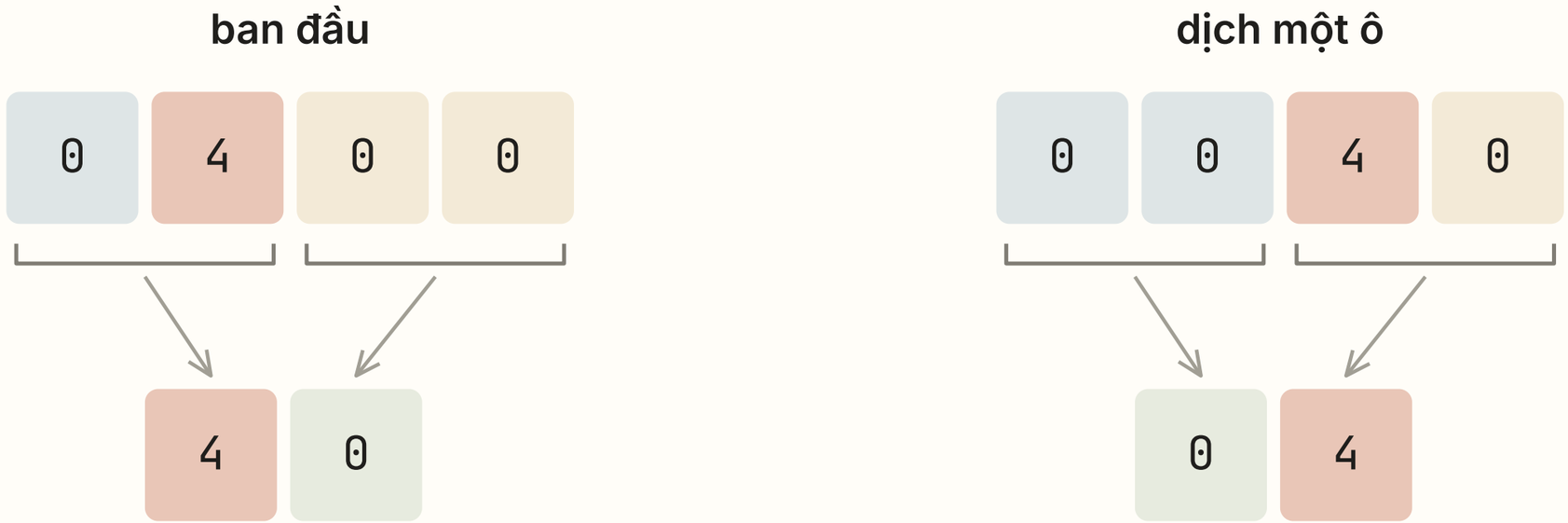
Ba lần max pool trên ảnh thật



mỗi lần max pool 2×2 giữ một số trong bốn: nét còn, vị trí chính xác mất dần

Bản đồ nét của ảnh thật qua ba lần max pool 2×2: từ 128×128 xuống 16×16, đường nét vẫn còn nhưng chi tiết và vị trí chính xác mất dần.

Pooling dịch vị trí



đỉnh qua ranh giới cửa sổ: kết quả đổi chỗ, pooling không bất biến dịch tuyệt đối

Max pool cửa sổ 2, stride 2 (số từ code bài): đỉnh 4 dịch một ô qua ranh giới hai cửa sổ thì đầu ra đổi từ (4, 0) thành (0, 4).

Dịch đỉnh qua ranh giới cửa sổ

```
left = torch.tensor([[[[0., 4., 0., 0.]]]])
right = torch.tensor([[[[0., 0., 4., 0.]]]])
pool = nn.MaxPool2d((1, 2))
print(pool(left).flatten().tolist())
print(pool(right).flatten().tolist())
```

KẾT QUẢ

```
[4.0, 0.0]
[0.0, 4.0]
```

Đỉnh 4 dịch sang cửa sổ thứ hai: pooling không bất biến dịch tuyệt đối.

HỎI NHANH

Max pooling trên một cửa sổ 2×2 chỉ trả về số 8. Từ đó có khôi phục duy nhất được bốn ô ban đầu không?

A Có, mỗi ô bằng $8/4 = 2$

B Có, nếu biết thêm stride của pooling

C Có, nếu biết thêm average pooling của cửa sổ

D Không, vì nhiều cửa sổ khác nhau có cùng giá trị lớn nhất

Gradient qua max pooling

```
window = y.detach().clone().requires_grad_(True)
nn.MaxPool2d(2)(window).sum().backward()
print(window.grad[0, 0].tolist())
```

KẾT QUẢ

```
[[0.0, 1.0], [0.0, 0.0]]
```

Max: chỉ ô thắng nhận gradient. Average: chia đều $1/4$.

HỎI NHANH

`MaxPool1d(kernel_size=2, stride=1)` trên `[1, 9, 3]` cho ra `[9, 9]`. Lấy tổng đầu ra rồi gọi `backward()`. Gradient về đầu vào là gì?

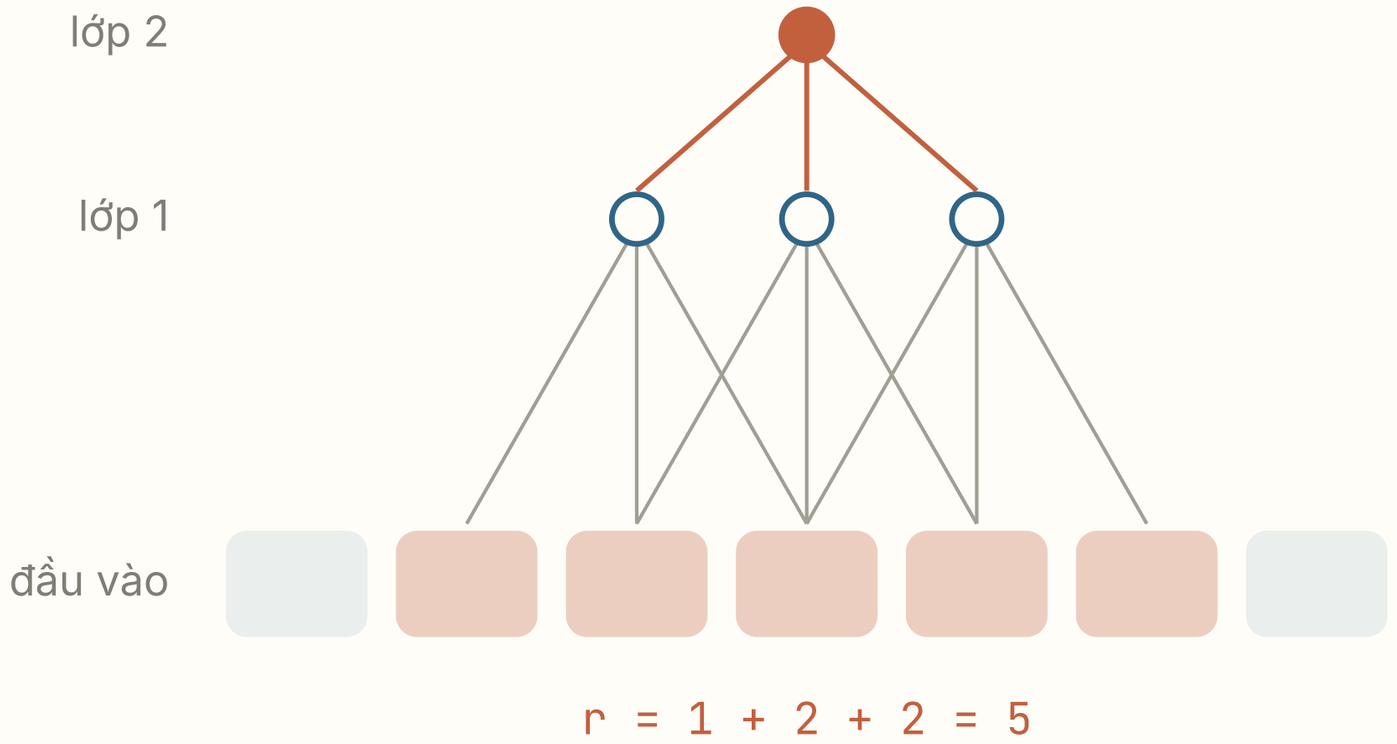
A `[0.5, 1, 0.5]`

B `[0, 9, 0]`

C `[0, 1, 0]`

D `[0, 2, 0]`

Hai lớp nhìn 5 ô



Hai lớp kernel 3, stride 1: ô ở lớp 2 đọc ba ô lớp 1, mỗi ô ấy đọc ba ô đầu vào chồng lấn nhau, nên vùng nhìn gồm 5 ô.

Theo dõi hai số

r: số ô input trong vùng nhìn; j: khoảng cách trên input giữa hai vị trí output kề nhau. Bắt đầu $r = j = 1$.

$$r_{\text{mới}} = r + (k-1)d\,j, \qquad j_{\text{mới}} = s\,j$$

Conv 3, stride 1: $r = 3$, $j = 1$. Pool 2, stride 2: $r = 4$, $j = 2$. Conv 3, stride 1: $r = 8$, $j = 2$; cả ba lớp dùng dilation 1.

Xếp chồng 3×3

Hai lớp 3×3 nhìn 5×5.

18 trọng số thay vì 25 mỗi cặp kênh, thêm một activation.

Lớp nông: map lớn, cạnh, nét.

Lớp sâu: map nhỏ, thô, tổ hợp lớn.

HỎI NHANH

Một conv kernel 3 stride 2, tiếp theo là một conv kernel 3 stride 1, cả hai có dilation 1. Một ô ở lớp thứ hai nhìn bao nhiêu ô đầu vào theo mỗi chiều?

A 5

B 6

C 7

D 9

04

Giả định và bẫy

Dịch thì theo, xoay thì không

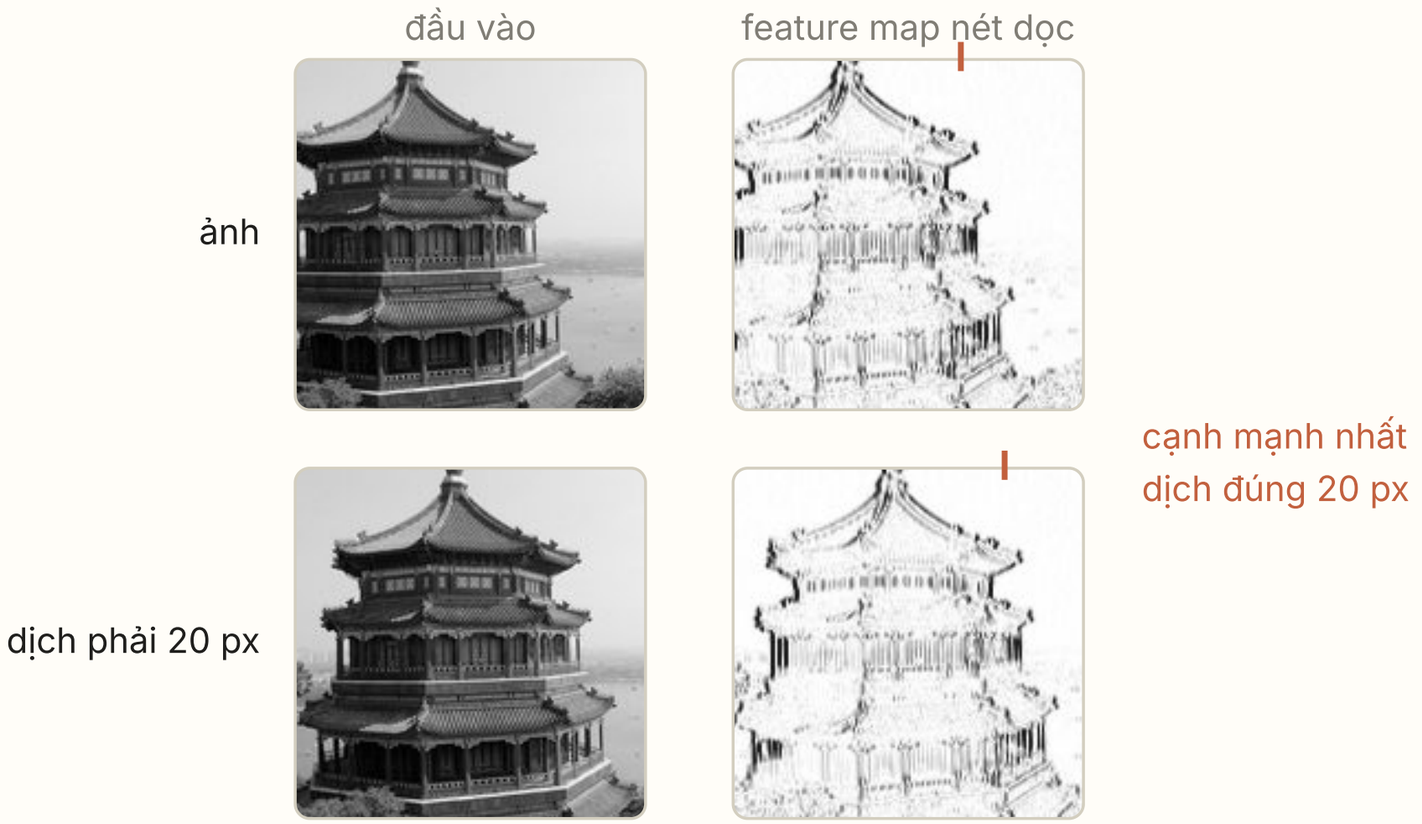
Conv stride 1: ảnh dịch, map dịch theo (đồng biến).

Bất biến chỉ có sau phép gộp vị trí.

Không tự bất biến với xoay, tỉ lệ, ánh sáng.

Đề cần tọa độ: đừng gộp hết vị trí.

Ảnh dịch, feature map dịch theo



Đồng biến với phép dịch trên ảnh thật: dịch ảnh sang phải 20 px thì feature map nét dọc dịch theo đúng 20 px, giá trị giữ nguyên trừ sát biên.

HỎI NHANH

Một bộ dò nét đã học tốt trên những ảnh chỉ có nét dựng đứng, ở đủ mọi vị trí. Ảnh test bị xoay 90° nên nét thành nằm ngang. Nhiều khả năng chuyện gì xảy ra?

A Nhận kém, vì kernel nét dọc phản ứng yếu với nét ngang

B Nhận kém, vì xoay làm đổi shape `(N, C, H, W)`

C Vẫn nhận tốt, vì dùng chung kernel ở mọi vị trí

D Vẫn nhận tốt, nếu thêm padding cho ảnh đã xoay

Bẫy shape

NumPy ảnh: (H, W, C).

PyTorch ảnh: (N, C, H, W).

Đổi trục bằng `permute`.

`reshape` không tự đổi ý nghĩa kênh.

Kiểm một pixel trước và sau.

HỎI NHANH

Ảnh NumPy `img` có shape `(32, 48, 3)`: cao 32, rộng 48. Viết `x = torch.from_numpy(img).permute(2, 1, 0)`, rồi đưa `x[None]` vào `nn.Conv2d(3, 8, 3)`. `x` có shape gì, và chuyện gì xảy ra?

A `(3, 32, 48)`, đúng thứ tự `(C, H, W)` cần có

B Báo lỗi ở `permute`, vì ảnh không vuông

C `(3, 48, 32)`, và `Conv2d` báo lỗi vì sai shape

D `(3, 48, 32)`, ảnh bị chuyển vị nhưng code vẫn chạy

Khi đi thi

- Batch giả: in shape sau mỗi khối.
- `(H, W, C)` → `permute` sang `(N, C, H, W)`.
- Downsample vừa đủ cho chi tiết nhỏ nhất.
- Cần tọa độ hay mask: giữ chiều không gian.
- Đo thời gian ở kích thước ảnh thật.

05

Luyện đề

LUYỆN ĐỀ 1

Một đội thay một conv 7×7, 64 kênh vào, 64 kênh ra, bằng ba conv 3×3 từ 64 sang 64 nối tiếp, stride 1, có ReLU ở giữa. So với lớp cũ, bỏ qua bias, điều nào đúng?

A Vùng nhìn chỉ còn 3×3, và ít trọng số hơn

B Vùng nhìn thành 9×9, và ít trọng số hơn

C Vùng nhìn vẫn là 7×7, và ít trọng số hơn

D Vùng nhìn vẫn là 7×7, nhưng nhiều trọng số hơn

PHỎNG THEO NOAI 2026 Singapore · Vòng sơ loại · Câu 215

LUYỆN ĐỀ 2

Một conv 1D có một bộ lọc, và z_j lớn nghĩa là có mẫu cần tìm ở vị trí j. Mạng A cho ra $\sigma(\max_j z_j)$, mạng B cho ra $\sigma(\min_j z_j)$. Đầu ra cao nghĩa là "mẫu có ở mọi vị trí" ở mạng nào, và nghĩa là "mẫu có ở ít nhất một vị trí" ở mạng nào?

A A rồi B

B B rồi A

C A ở cả hai

D B ở cả hai

PHỎNG THEO MIT 6.036 · Final, Spring 2022 · Q8

LUYỆN ĐỀ 3

Mỗi mẫu là một tensor `(64, 64, 2)`: vector lực (F_x, F_y) đo trên lưới 64×64. Phải dự đoán tọa độ nguồn (x_s, y_s) cùng 100 giá trị cường độ. Một đội dùng các conv stride 1, rồi global average pooling, rồi `Linear` ra 102 số, và lỗi tọa độ rất cao. Thay đổi nào nhắm đúng nguyên nhân?

A Đổi global average pooling thành global max pooling

B Tăng gấp đôi số kênh của lớp conv cuối

C Thêm padding để conv đọc được cả vùng biên

D Bỏ global pooling, flatten feature map để còn giữ vị trí

PHỎNG THEO USA-NA-AIO 2026 · Vòng 2 · Ngày 1 · Problem 2

Chốt ý

Cửa sổ nhỏ và trọng số chung giúp dò cùng một tín hiệu ở nhiều vị trí.

Pooling lấy biểu diễn gọn hơn nhưng làm mất một phần chi tiết vị trí.

Sau giờ học

Làm **10 câu kiểm tra cuối bài** trên trang bài học.

Chạy notebook; đổi stride và padding, đoán trước shape đầu ra rồi kiểm lại.